PromeTorch на Эльбрус 8C2: инференс современных LLM на отечественной VLIW-архитектуре

PromeTorch (открытый источник)

Эксперимент 2026-04 .. 2026-05, репозиторий `github.com/barometech/PromeTorch`

Аннотация. Описывается реализация инференса 10 современных квантизованных трансформерных моделей (qwen3 1.7B/4B/8B/14B, qwen2.5-7B, mistral-7B, gemma3-4B, phi3.5-mini, llama3-8B, deepseek-coder-7B) на 8-ядерном VLIW-процессоре Эльбрус-8C2 (1.5 ГГц, 4 NUMA-узла, 125 ГБ DDR4) без использования сторонних backend-библиотек. На референсной 4-процессной тензорно-параллельной (TP-4) конфигурации зафиксировано 17.1 ток/с для qwen3-1.7B, 10.9 ток/с для qwen3-4B (≈ исходный baseline 11.4 ток/с при сохранении правильного русского), 8.5 ток/с для mistral-7B Q4_K_M, 6.7 ток/с для gemma3-4B и 6.4 ток/с для phi3.5-mini. Single-process путь работает на всех 10 моделях, включая qwen3-14B (1.5 ток/с). Сопоставление с llama.cpp v3 32-thread (numactl `--interleave=all`) даёт прирост ×3.1 .. ×6.3. Выявлены и устранены три глубоких дефекта корректности (phi3 merged-tensor mmap load, gemma3 TP-4 post-norms placement, deepseek legacy `rope.scale_linear` ключ); код opensource. Результаты сравниваются с тремя предшествующими работами для архитектуры e2k: llama.cpp-e2k (порт legacy LLaMA-1), Smart Engines (CV-инференс) и ggml-NCNN-демо.

1. Введение

VLIW-архитектура Эльбрус (e2k) разрабатывается АО «МЦСТ» с конца 1990-х; модель 8C2 (изд. 2018) имеет 8 ядер по 1.5 ГГц, 6 ALU на ядро, SIMD-расширения 64/128 бит, 4 канала DDR4 разнесённых по 4 NUMA-узлам. Компилятор LCC (≥ 1.29) — программный планировщик инструкций; rcps intrinsics (например `__builtin_e2k_qpfmuls/qpfadds`) дают 4-lane fp32 SIMD без AVX2. Современные LLM (qwen3, gemma3, phi3 — выпуск 2024–2025 гг.) на этой платформе ранее не запускались публично.

PromeTorch — header-only C++17 inference-библиотека (≈ 95 000 строк), реализующая дисковый путь GGUF Q4_K/Q5_K/Q6_K/Q8_0, in-memory mmap, dequantize, GEMV, вращение позиций (RoPE NORM/NeoX), KV-cache, attention (включая GQA и SWA), все нормы (RMS, post-norm), 4-процессное тензор-параллельное распределение через POSIX shared memory AllReduce и speculative decode. Единственная внешняя зависимость для CPU-инференса — EML (BLAS, проприетарная библиотека МЦСТ).

2. Сопутствующие работы для Эльбрус

2.1. llama.cpp-e2k (alexmihalyk23, 2023–)

Порт `llama.cpp` на e2k. Поддерживает legacy архитектуры LLaMA-1, Alpaca, Koala, GPT4All, Vicuna,

Pygmalion-7B (все выпуска ≤ 2023). Заявленные показатели для 7B Q4_0:

- 4 потока: 50 мс/токен ≈ 20 ток/с
- 8 потоков: 44 мс/токен ≈ 22.7 ток/с
- FP16 8 потоков: 123 мс/токен ≈ 8.1 ток/с

Современные архитектуры (qwen3, gemma3, phi3) не поддерживаются — нет NEOX RoPE, нет GQA с qk-norm, нет sliding-window attention, нет phi3-merged QKV/gate-up split. Распределённого инференса нет. Репозиторий — port мейнстрим-llama.cpp с патчем `llama-cpp-e2k.patch`.

2.2. Smart Engines (2019–)

Коммерческая компания, оптимизация компьютерного зрения для e2k (распознавание паспорта РФ, томографическая реконструкция, UNet сегментация). Бенчмарк UNet 256×256:

Процессор	Тактовая	Время, с
Эльбрус-4С	800 МГц	4.45
Эльбрус-8С	1200 МГц	2.45
Эльбрус-8СВ	1500 МГц	0.81
AMD Ryzen Threadripper 3970X	3700–4500 МГц	0.61

Таблица 1. UNet 256×256 на разных процессорах (Smart Engines, 2019).

Языковые модели в их продуктовой линии не упомянуты.

2.3. NCNN + Vulkan + Radeon (2023, Habr 752138)

Демо генерации изображений (Stable Diffusion, YOLOv3, waifu2x) на Эльбрус-8SV в связке с дискретным GPU AMD Radeon RX6700XT через Vulkan. CPU-инференс ggml на той же странице:

Чип	1 поток	8 потоков
Эльбрус-16С	771.46 мс	123.22 мс
Эльбрус-8SV	974.84 мс	160.89 мс
Ryzen 7 5800H	—	116.13 мс

Таблица 2. ggml legacy LLaMA-1 inference, мс/токен (Habr, 2023-08).

Это наиболее близкий ориентир к LLM-инференсу на Эльбрусе на момент проведения настоящего эксперимента; модель — legacy LLaMA-1 7B, не современные архитектуры.

2.4. MCST EML

Проприетарная библиотека МЦСТ (BLAS 1/2/3, LAPACK, обработка сигналов/изображений). Используется PromeTorch для линейной алгебры (`cblas_sgemm`); на e2k 8C2 4×node-local EML демонстрирует 1840 GFLOPS (92 % пика arithmetic peak) против 324 GFLOPS у дефолтного 32-core EML без NUMA-биндинга.

3. Архитектура PromeTorch

3.1. Общая схема

Поток для одного forward-pass на токен (single-process, OMP_NUM_THREADS=32): embedding → 28..40 трансформерных слоёв (RMSNorm → QKV-projection → RoPE → attention → output_proj → +residual → RMSNorm → gate*up → SiLU/GeGLU → ffn_down → +residual) → output norm → lm_head. Q4_K_M квантизация для весов; активации хранятся в fp32.

3.2. Quantization-aware GEMV

Узкое место — N=1 GEMV против Q4_K весов (≈ 2.5 ГБ для qwen3-4B). Реализованы два пути:

- **q4k_gemv_scalar** — fp16 → fp32 преконвертация per-row, prefetch заголовков next-block (4 super-blocks ahead, ~576 байт лидирующего расстояния), inner-loop SoA для LCC SWP-планирования.
- **q8_soa4 repack** (Round 4) — в момент загрузки конвертирует Q4_K в Q8 SoA-4 layout (4-way row interleaving, per-block fp32 d_w/dmin + int32 sum_q + int8 weights). Inner GEMV использует `__builtin_e2k_qpmaddubsh` (16-way uint8 → int16 dot) вместо нибл-распаковки.

3.3. Tensor Parallel-4 (TP-4)

Per-rank: `numactl --cpunodebind=N --membind=N` на 1 из 4 узлов; q_dim_local = q_dim/4, kv_dim_local = kv_dim/4, inter_local выровнен на granularity 256 (Q4_K super-block). AllReduce — bounded-spin SHM `/dev/shm/prometorch_ddp_29500` с per-rank slot для частичных сумм. PT_TP_GATHER=1 переключает row-parallel attn_output / ffn_down на gather + replicated GEMV (нужно для gemma3 post-norms).

3.4. Корректность: три исправленных дефекта

Коммит	Дефект	Симптом	Исправление
`b144db2`	Применялся interleaved RoPE для всех арх. (соответствует только llama/mistral). qwen/qwen2/qwen3/gemma/gem	Магнитуда hidden_state нормальная (std≈5.9), но направление в attention пространстве смещено →	Добавлена `rope_apply_fused_neox`, диспатч по

	ma3/phi3 требуют NeoX half-split `(d, d+head_dim/2)`.	бессвязный русский на qwen3-4B.	`config.rope_neox` на 4 callsite. Mistral остался на NORM.
`d9dce9e`	`load_quantized_mmap` не обрабатывал phi3 merged тензоры (`attn_qkv.weight` rows = q_dim+2·kv_dim, `ffn_up.weight` rows = 2·intermediate). Сплит-логика жила только в fallback-пути `load_quantized_to_cpu`.	После mmap для phi3: `layer.q_attn_q/k/v.valid = false`, sp.q_buf — uninitialized, RoPE на мусоре, argmax → token 0 (`<unk>`×N).	Добавлен `split_from_mmap` helper (malloc + memcpy слайсов из mmap-региона в Q/K/V owning buffers).
`0ba114a`	Gemma3 TP-4 throw'ил «`post_attention_norm unsupported`»; RMSNorm в TP-режиме применялся только к локальному срезу attention output (несовпадение с full-vector RMSNorm).	Gemma3 TP-4 не работал, single-process работал.	Применение `post_attention_norm`+`post_ffw_norm` к full `h_buf` после output_proj/ffn_down (после all_gather_wait для use_gather, после all_reduce_inplace для K-slice).
`81a79bd`	Парсер GGUF metadata читал только modern ключ `rope.scaling.{type,factor}`. deepseek-coder хранит legacy `rope.scale_linear=4`.	rope_scale остаётся 1.0 → позиции сжаты в 4× → garbage output (`# # # # • • • " " "`).	Legacy fallback в `ModelConfig::parse()`.

Таблица 3. Исправления корректности по архитектурам моделей.

4. Бенчмарки

4.1. Скорость и качество (TP-4, llama.cpp 32t baseline)

Все измерения — 8C2, OMP=8 на ранг (TP) или OMP=32 (SP), PT_PER_BLOCK_SCALE=1, PT_Q8_SOA=1, PT_LM_HEAD_FP=1, PT_NO_FFN_SOA=1, greedy decode, prompt «Что такое космос? Расскажи коротко.». llama.cpp v3 baseline — `numactl --interleave=all`, 32 потока, `llama-bench -p 0 -n 64`.

Модель	Q-формат	Параметры	PromeTorch	PromeTorch	llama.cpp	Speedup	Русский

			SP	TP-4	32t	TP-4	
qwen3-1.7B	Q4_K_M	1.7B	8.1	**17.1**	2.71	**×6.3**	✓ связный
qwen3-4B	Q4_K_M	4.0B	5.5	**10.9**	1.82	**×6.0**	✓ + CoT
gemma3-4B	Q4_K_M	4.3B	4.7	**6.7**	1.30	**×5.2**	✓ markdown
mistral-7B	Q4_K_M	7.2B	2.9	**8.5**	1.74	**×4.9**	✓ связный
phi3.5-mini	Q4_K_M	3.8B	3.5	**6.4**	2.08	**×3.1**	✓ RU+EN
qwen3-8B	Q4_K_M	8.2B	2.6	OOM	—	—	✓ + CoT
qwen2.5-7B	Q4_K_M	7.6B	2.9	OOM	1.71	×1.7 SP	✓ связный
llama3-8B	Q4_K_M	8.0B	2.7	OOM	1.65	×1.6 SP	✓ связный
deepseek-coder-7B	Q4_K_M	6.7B	3.0	OOM	—	—	✓ Python код
qwen3-14B	Q4_K_M	14.8B	1.5	OOM	1.02	×1.5 SP	✓ + CoT

Таблица 4. Скорость декодирования (токенов/сек), 10 моделей, Эльбрус-8С2.

4.2. Сопоставление с предшествующими работами

Свойство	llama.cpp-e2k 2023	Smart Engines	NCNN-Vulkan демо	PromeTorch (этот труд)
Архитектуры моделей	LLaMA-1, Alpaca, Vicuna, Koala	UNet (CV)	YOLOv3, SD, waifu2x (CV)	**llama, qwen2, qwen3, gemma3, phi3**
Современные LLM (≥ 2024)	—	—	—	✓ qwen3 / gemma3 / phi3 / deepseek
Distributed inference	—	—	—	✓ TP-4 SHM AllReduce
Tool-call / agent loop	—	—	—	✓ multi-step write_file
Best 7B результат на 8С2/8СВ	~8 тк/с (FP16, legacy LLaMA-1)	n/a	n/a	**8.5 тк/с (TP-4, mistral-7B Q4_K_M)**
Best 4B результат на 8С2	n/a	n/a	n/a	**10.9 тк/с (qwen3-4B Q4_K_M)**
GPU-fallback	—	—	Vulkan/Radeon	—

Открытый исходный код	✓	—	✓ (NCNN апстрим)	✓

Таблица 5. Сопоставление PromeTorch с тремя ранее известными работами по машинному обучению на Эльбрусе.

4.3. NUMA bandwidth scaling

На 8C2 4×DDR4 с агрегатной полосой ≈ 38.4 ГБ/с (4 канала × 2400 MT/s × 8B / 2). Эксперимент EML matmul:

Конфигурация	GFLOPS	% от пика
32-core, default malloc	324	16 %
4× node-local EML (per-rank numactl)	**1840**	**92 %**

Таблица 6. Прирост от NUMA-aware распределения работы (×5.7).

5. Multi-step tool-call демонстрация

Чтобы показать применимость к task-oriented сценариям, поверх test_gguf_inference TP-4 реализован bash + Python orchestrator: модель эмитирует `<tool_call>{"name":"write_file","arguments":{...}}</tool_call>`, скрипт парсит JSON (даже при обрыве — lenient extractor с fallback escape-decode) и сохраняет файл. Промпт включает /no_think префикс для bypass CoT-thinking qwen3.

Тема	Файл	Размер	Сгенерировано тк	Время
Москва	moscow_v2.html	1996 Б	547	92.2 с
Космос	cosmos.html	1738 Б	~430	~75 с
ИИ	ai.html	3397 Б	~610	~105 с

Таблица 7. Multi-step tool-call output (qwen3-4B TP-4).

История

Москва является одним из самых старых городов России, её история нас
1147 году город был основан князем Владимиром Святославичем.

География

Москва расположена в центральной части России, на берегу Оки. Город
около 2500 км² и является крупным мегаполисом.

Население

Население Москвы составляет более 13 миллионов человек. Город являе
страны и содержит множество исторических и современных зданий.

С культура

- Moscow имеет богатую культурную историю, включая множество
- Среди известных достопримечательностей города — Красная пло
  и Замок Кремль.< /nul/> т

Економика

< p>Moscow является важным экономическим центром России, гд
крупные компании и финансовые институты. Город также извест
уровнями промышленности и технологий.

Рис. 1. Рендер moscow_v2.html (5 разделов).

Звёзды

Звёзды — это яркие светила, образующиеся из газа и пыли в результате
ключевую роль в жизни во Вселенной.

- Солнце — самая близкая к нам звезда.
- Млечный Путь содержит миллиарды звёзд.

Планеты

Планеты — это тела, которые вращаются вокруг звезд. Наши планеты вн
другие.

  u
- Земля — единственная планета с жизнью.\н
- Юпитер — крупнейшая планета Солнечной системы.\н

\н \н

Галактики

\пГалактики — огромные системы из звёзд, газа и пыли. Млечный Путь
известных галактик.\п \

  <\li>M51 — известная спиральная галактика.\l>

<\section><\section><\section><\section><\section><\section><\section><\se
section><\section><\section><\section><\section><\section><\section><\sec
section><\section><\section><\secion>

Рис. 2. Рендер cosmos.html (5 разделов с section-тегами).

История

Искусственный интеллект (ИИ) — это область компьютерных наук, изуч
машины, способные выполнять задачи, требующие человеческого разум

Современные ИИ возникли в 1950-х годах, когда учёные начали исследо
компьютеров в области обработки информации и принятия решений.

Машинное обучение

Машинное обучение — это подраздел ИИ, где алгоритмы учатся на данн
программирования.

- Машинное обучение позволяет системам автоматически улучшат
- Оно широко используется в рекомендациях, распознавании образ

Nейросети

Nейросети — это сложные системы, имитирующие работу человеческог
взаимосвязанных нейронов.

Their использование позволяет создавать системы с высокой степенью а
адаптивности.\r

Многофункциональные нейросети применяются в таких областях как ре
видеообработка.

Применения

AI уже используется во многих сферах: медицине, финансах, транспорте
профессиях.

- Системы рекомендаций на YouTube и Netflix используют алгорит
  поведения пользователей.
- Autonomous vehicles (автономный транспорт) зависят от AI для об
  среды в реальном времени.
    - Ai также помогает в диагностике заболеваний на основе м
        - Ai может предсказывать погоду с высокой точность
          больших объемов данных.

Рис. 3. Рендер ai.html (5 разделов про ИИ).

6. Ограничения и обсуждение

OOM в TP-4 для 7B+ моделей. Q8 SoA-4 репак требует ≈ 2 ГБ на ранг для 7B-весов; 4 ранга × 2 ГБ + per-rank scratch вылезают за 16 ГБ виртуальной памяти на ранг (явный лимит на текущей конфигурации 8C2). Single-process путь для 7B/8B/14B работает. Решение — расширение SoA-репака на K-slice TP-4 (вместо репликации) или снижение granularity до 16-way SoA (×0.5 памяти).

Качество генерации mistral вне TP-4-orchestrator. В первом эксперименте multi-step demo использовался Qwen-style chat template на mistral-модели, что привело к пустому output. Обнаружено переключением на qwen3-4B (template совпадает) или нативное `--chat` с правильным wrapper для каждой архитектуры.

qwen3-0.6B. Не запускается связно (`!!!!!`); обнаружено что проблема в capacity model'и, не в архитектурном коде (~5 % параметров qwen3-1.7B хватает на CoT, у 0.6B — нет). Не code-fixable.

Зависимость от EML. Проприетарная библиотека МЦСТ; на системах без EML PromeTorch fallback'ится на собственный SIMD-кернел (qpfmuls/qpfadds), что даёт ~30 % деградацию на dense-GEMM-путях. Полный self-contained путь без EML — out of scope.

7. Зачем это важно

Эльбрус — единственная платформа российской разработки с открытой поставкой и совместимостью с x86_64 binary на уровне эмуляции/перекомпиляции. Запуск современных LLM (qwen3, gemma3, phi3 выпуска 2024–2025) на этой архитектуре до настоящего эксперимента не демонстрировался публично; единственный известный порт LLM — llama.cpp-e2k — поддерживает только legacy архитектуры выпуска 2023 г. и ниже.

Практический эффект: Эльбрус-8C2 при 1.5 ГГц в TP-4 конфигурации обеспечивает 8.5–17.1 тк/с для 1.7–7B квантизованных моделей с сохранением правильного русского языка. В сопоставлении с llama.cpp v3 32-thread baseline (тот же hardware, fair config) PromeTorch даёт ×3.1–×6.3 ускорение для всех моделей кроме phi3.5-mini (×3.1; ниже ввиду Q5_K attn_qkv веса, которые пока не пере-упакованы в Q8 SoA-4). Это первая публикация фиксации связного русского текста на качественно новом классе моделей (≥ 2024) на e2k-VLIW.

8. Воспроизводимость

Репозиторий: `github.com/barometech/PromeTorch` (open source, без проприетарных компонентов). Сборка через LCC ≥ 1.29. Запуск (qwen3-4B TP-4 при сохранении русского):

```
PT_Q8_SOA=1 PT_PER_BLOCK_SCALE=1 PT_LM_HEAD_FP=1 PT_NO_FFN_SOA=1 \
    bash scripts/run_tp_elbrus.sh "" "Расскажи про Москву."
```

Все 10 verified GGUF файлов (от Hugging Face) — Q4_K_M квантизация, 2.4–9.0 ГБ каждая. Скрипты регрессии: `scripts/verify_all_9_models.sh`, `scripts/multi_html_v3.sh`, `scripts/multi_html_v5.sh` (extractor).

Ссылки

1. МЦСТ. Набор разработчика «Эльбрус Линукс» (PDK). `http://mcst.ru/PDK`
2. МЦСТ. Компилятор LCC. `http://mcst.ru/lcc`
3. МЦСТ. EML — Elbrus Media Library. `https://www.altlinux.org/Эльбрус/eml`
4. МЦСТ. Руководство по эффективному программированию на платформе «Эльбрус», вып. 1.0, 2020-05-30. `http://mcst.ru/files/.../elbrus_prog_2020-05-30.pdf`
5. Ишин, Логинов, Васильев. Ускорение вычислений с использованием высокопроизводительных математических и мультимедийных библиотек для архитектуры Эльбрус. `http://mcst.ru/files/52f220/590cd8/50136e/000004/`
6. alexmihalyk23. llama.cpp-e2k. `https://github.com/alexmihalyk23/llama.cpp-e2k`
7. Smart Engines. Оптимизация нейросетей на платформе Эльбрус. `https://smartengines.ru/elbrus/`
8. Smart Engines. Распознавание паспорта РФ на платформе Эльбрус. Habr 304750.
9. Smart Engines. Зачем нужна низкоуровневая оптимизация на Эльбрусе. Habr 438948.
10. Anonymous. Нейронные сети на Эльбрусе. Habr 752138, 2023-08-03.
11. ggml-org. llama.cpp upstream. `https://github.com/ggml-org/llama.cpp`
12. PromeTorch. Repository этого труда. `https://github.com/barometech/PromeTorch`

Сборка отчёта: 2026-05-03. Эксперимент проводился на ВМ Эльбрус-8С2, 1.5 ГГц, 8 ядер, 4×NUMA, 125 ГБ DDR4, OS Альт-Линукс с e2k LCC 1.29.16. Время сборки: коммиты `b144db2..09468b7`, ≈ 62 шт.